A Inteligência Artificial no Sistema de Rastreamento de Alvos

Impacto, origem e caminhos de melhoria do controlador neuroevolutivo

Documento explicativo do projeto

27 de outubro de 2026

Sumário

1 Resumo em uma página **2**

2 De onde essa IA veio **2**

2.1 O algoritmo original (sem IA nenhuma) 2

2.2 Por que neuroevolução, e não outro tipo de IA 2

2.3 Linha do tempo (pelo histórico de commits do repositório) 3

3 O que a IA realmente decide (e o que ela nunca decide) **4**

3.1 As seis entradas e duas saídas da rede 4

4 Como ela é treinada **4**

5 Limitações atuais (honestas) **5**

6 Como melhorar essa mecânica **5**

6.1 Ganhos de curto prazo (baixo risco, poucas linhas de código) 5

6.2 Ganhos de médio prazo 6

6.3 Ganhos de longo prazo (mudam a arquitetura do sistema) 6

7 Glossário rápido **6**

1 Resumo em uma página

O sistema tem, hoje, **dois cérebros possíveis** para decidir qual torre persegue cada alvo: uma heurística pura (regras fixas de prioridade por quadrante) e uma heurística *combinada com uma rede neural pequena*, treinada por um algoritmo genético dentro do próprio aplicativo. A rede neural **nunca substitui** a heurística — ela só ajusta, dentro do que a heurística já considerou uma escolha razoável, qual candidata específica vence e em que instante exato a torre deve disparar. Se nenhuma rede treinada estiver embutida no aplicativo, o sistema cai de volta para a heurística pura (menor distância), sem travar nem mudar de comportamento de forma imprevisível.

Essa rede é treinada numa tela dedicada ("Treinamento do Controlador"), dentro de um mundo simulado em 2D — nunca com o hardware real. O resultado do treinamento (um conjunto de números, chamado *genoma*) pode ser exportado, embutido no instalador do aplicativo e, opcionalmente, sincronizado por porta serial para uma cópia somente leitura rodando dentro do próprio firmware do Arduino (hoje usada só para diagnóstico/comparação, não para decidir nada sozinha no microcontrolador).

2 De onde essa IA veio

2.1 O algoritmo original (sem IA nenhuma)

O projeto nasceu com um algoritmo de seleção de torre inteiramente baseado em regras escritas à mão (`Services/TowerSelectionService.cs`), descrito em detalhe no documento *Algoritmo de Seleção de Torre — Fundamentação Matemática*:

1. Filtra torres disponíveis e livres;
2. Prioriza a torre do **próprio quadrante** do alvo;
3. Se ela estiver ocupada, usa uma torre de quadrante **adjacente** (cooperação);
4. Se nenhuma das duas existir, usa qualquer torre livre restante (**cobertura**);
5. Em último caso, reaproveita a torre mais próxima mesmo já comprometida (**sobrecarga**), para nunca deixar um alvo sem nenhuma torre associada;
6. Dentro da camada escolhida, a candidata vencedora era simplesmente a de **menor distância Euclidiana** até o alvo.

Esse algoritmo é determinístico, auditável e continua existindo **integralmente** — é a estrutura de camadas (casa → adjacente → cobertura → sobrecarga) que organiza a decisão até hoje. A IA entrou depois, para resolver um problema específico: dentro de uma camada com mais de uma candidata, "menor distância" é uma regra razoável, mas não é obrigatoriamente a melhor — por exemplo, pode ignorar que um alvo está muito rápido, ou que o ângulo de abordagem dele vai dificultar a mira de uma torre em particular.

2.2 Por que neuroevolução, e não outro tipo de IA

A escolha foi por algoritmo genético evoluindo uma rede neural pequena (*neuroevolução*), em vez de, por exemplo, aprendizado por reforço clássico ou uma rede treinada por retropropagação

supervisionada, por três razões registradas no próprio código e nos artigos de embasamento do TCC:

- **Não precisa de dados rotulados.** Não existe um conjunto de "decisões corretas" gravado previamente — o sistema não sabe, a priori, qual é a torre ideal para cada situação. O algoritmo genético só precisa de uma função de qualidade (*fitness*: alvos abatidos, escapados e tiros errados), não de exemplos prontos.
- **Auditável.** Um algoritmo genético evoluindo pesos de uma rede pequena (6 entradas, 8 neurônios ocultos, 2 saídas) é simples de implementar do zero e de explicar numa banca de TCC, sem depender de bibliotecas de IA de caixa-preta.
- **Hibridização natural com a heurística existente.** A rede neural só entra *dentro* da estrutura de camadas já escrita à mão — ela nunca decide sozinha algo que a heurística já não tivesse considerado possível. Isso elimina o risco clássico de uma IA pura: escolhas absurdas (torre ocupada, ignorar cooperação entre quadrantes) continuam impossíveis por construção, não por a rede "ter aprendido" a evitá-las.

2.3 Linha do tempo (pelo histórico de commits do repositório)

- Implementação do `TowerSelectionService` puramente heurístico, com as camadas casa/adjacente/cobertura/sobrecarga.
- Introdução do pacote `NeuralEvolution`: `RedeNeural`, `AlgoritmoGenetico`, `ControladorHibrido` e o ambiente simulado `AmbienteTreinamento`, junto com a tela "Treinamento do Controlador".
- Treinamento passa a usar o alcance de mira e a certeza de disparo configurados pelo usuário em vez de valores fixos no código — a simulação fica mais fiel às configurações reais do sistema.
- Seis regras heurísticas adicionais (anti-oscilação, urgência, corredor de tiro, balanceamento de carga, predição de trajetória, ajuda de torre adjacente) são incorporadas tanto no sistema real quanto na simulação de treinamento, para o genoma treinar contra o comportamento mais próximo possível do produtivo.
- `ControladorTreinadoProvider` embute o melhor genoma exportado como recurso do instalador; `RedeEmbutidaSincronizadorService` permite enviar esse mesmo genoma, peso por peso, por porta serial para uma cópia somente leitura dentro do firmware do Arduino (diagnóstico/comparação).
- Persistência de histórico de gerações, marcos de fitness a cada bloco de gerações e instantâneos de comportamento do melhor genoma (`IHistoricoTreinamentoRepository`, `IMarcoFitnessTreinamentoRepository`, `IInstantaneoComportamentoRepository`) — para acompanhar se a evolução é real ao longo de muitas sessões de treinamento, não só dentro de uma execução.
- Mais recentemente: catálogo de casos táticos nomeados (enxame/saturação, disputa pela mesma torre, alvo rápido, certeza baixa persistente, alvo perto da base, zona morta) somados à avaliação de toda geração, e um editor de **cenário personalizado** na própria tela, onde o usuário desenha à mão a posição e a velocidade de cada alvo e observa como o melhor genoma atual reage — sem nenhum sorteio.

3 O que a IA realmente decide (e o que ela nunca decide)

A rede neural influencia	**A rede neural nunca decide**
Qual torre, *dentro da camada já escolhida pela heurística*, persegue um alvo	Se uma torre pode ser escolhida fora das camadas casa/adjacente/cobertura/sobrecarga
Em que instante exato, dentro da janela já autorizada, a torre dispara	Se o disparo é autorizado — zona morta e certeza mínima são sempre checadas antes, fora do alcance da rede
O ajuste fino de pontuação entre candidatas empatadas na heurística	O acionamento automático do canhão — `FireControlService` continua exigindo confirmação humana (manual) para o disparo físico de fato ocorrer

Em outras palavras: **a IA refina uma decisão, ela não a inventa do zero**. Essa é uma escolha de projeto deliberada, não uma limitação técnica — está documentada explicitamente no código (`ControladorHibrido`, `TowerSelectionService`) como garantia de que o sistema nunca produz uma decisão fisicamente absurda só porque a rede "aprendeu algo estranho".

3.1 As seis entradas e duas saídas da rede

A rede neural (arquitetura $6 \to 8 \to 2$, totalmente conectada) recebe sempre as mesmas seis informações sobre o par alvo/torre candidata:

1. Distância normalizada até o alvo;
2. Se a torre é do mesmo quadrante do alvo;
3. Se a torre é de quadrante adjacente;
4. Ângulo de mira normalizado;
5. Velocidade do alvo normalizada;
6. Um fator numérico da camada heurística atual (1,0 = casa, 0,5 = adjacente, 0,0 = cobertura).

E produz duas saídas: um ajuste de pontuação (somado à pontuação heurística de distância) e uma confiança de disparo (se positiva, autoriza considerar o disparo — sempre depois dos critérios inegociáveis de zona morta e certeza mínima).

4 Como ela é treinada

O treinamento acontece inteiramente dentro de um mundo simulado 2D (`NeuralEvolution/AmbienteTreinamento.cs`), nunca contra o hardware físico:

1. Uma população de 60 "genomas" (conjuntos aleatórios de pesos da rede) é criada.

2. Cada genoma é testado em vários episódios simulados — alvos nascem na borda de um mundo circular e avançam em direção à base; o genoma controla quais torres perseguem quais alvos e quando disparam.

3. Cada episódio gera uma nota (*fitness*): pontos por abate, menos pontos por fuga e por disparo errado.

4. O algoritmo genético seleciona os melhores genomas (seleção por torneio), cruza pares deles e aplica mutação aleatória, produzindo a próxima geração — e repete, geração após geração, o mais rápido possível em segundo plano.

5. A cada geração, além de episódios aleatórios, o genoma é obrigatoriamente testado contra **todos** os casos táticos catalogados (enxame, disputa de torre, alvo rápido, certeza baixa, alvo perto da base, zona morta) — para a evolução não ignorar um caso difícil só porque o sorteio raramente produz aquela situação.

6. O usuário pode, a qualquer momento, inspecionar visualmente como o melhor genoma atual se comporta: rodando um episódio aleatório, escolhendo um caso catalogado específico, ou desenhando um **cenário totalmente personalizado** (posição X/Y em metros e velocidade de cada alvo, sem sorteio nenhum).

5 Limitações atuais (honestas)

- **O treinamento nunca vê dados reais.** Todo o aprendizado acontece num mundo simulado com física simplificada (alvos em linha reta, erro de disparo probabilístico por fórmula fixa). Não existe hoje um laço de realimentação que compare decisões da rede contra o que de fato aconteceu com o hardware físico.

- **Sem validação cruzada formal.** O fitness de cada geração usa seeds (sementes de sorteio) diferentes a cada geração, o que reduz o risco de a rede "memorizar" um cenário específico, mas não existe hoje um conjunto de episódios reservado exclusivamente para medir se a rede generaliza bem ou só se ajustou aos cenários de treino.

- **Arquitetura fixa.** A rede sempre tem exatamente 8 neurônios na camada oculta — esse número nunca foi comparado experimentalmente contra alternativas (4, 16, duas camadas ocultas etc.).

- **Genoma embutido é estático até a próxima build.** O Arduino recebe uma cópia somente leitura do genoma, só para diagnóstico — não há aprendizado nem ajuste nenhum acontecendo no hardware em campo.

6 Como melhorar essa mecânica

6.1 Ganhos de curto prazo (baixo risco, poucas linhas de código)

- **Conjunto de validação separado do conjunto de treino.** Reservar algumas seeds (e os cenários personalizados mais representativos criados na tela) para medir o fitness do melhor genoma *sem* elas influenciarem a seleção/mutação — um sinal direto de overfitting, hoje inexistente.

- **Comparar genomas de gerações diferentes lado a lado.** A tela já grava instantâneos de comportamento a cada bloco de gerações (`InstantaneoComportamentoMelhorGene`) — falta uma visualização que compare dois desses instantâneos diretamente (taxa de acerto, cobertura, regras aprendidas) para decidir se vale a pena substituir o genoma embutido em produção.
- **Mais cenários táticos catalogados.** O catálogo atual cobre seis casos; vale mapear outros, como alvo que desacelera/acelera no meio do trajeto, ou disparo simultâneo de três torres no mesmo alvo rápido.

6.2 Ganhos de médio prazo

- **Fitness multiobjetivo.** Hoje o fitness é um único número (abates menos fugas e menos erros, com pesos fixos). Técnicas como NSGA-II permitiriam otimizar abates e economia de tiros (menos erros) como objetivos separados, em vez de depender de um peso arbitrário escolhido à mão.
- **Busca de arquitetura.** Testar sistematicamente variações de neurônios/camadas ocultas (por exemplo, treinar 8, 12 e 16 neurônios em paralelo e comparar o fitness médio de validação) em vez de manter 8 fixo por convenção histórica.
- **Currículo de dificuldade.** Começar o treinamento só com cenários fáceis (um alvo lento) e introduzir gradualmente os cenários catalogados mais difíceis — pode acelerar a convergência comparado a já começar misturando todos os casos desde a geração 1.

6.3 Ganhos de longo prazo (mudam a arquitetura do sistema)

- **Laço de realimentação com o hardware real.** Registrar, durante operação física, qual torre foi escolhida, se o disparo acertou ou errou, e usar esses episódios reais (ainda que poucos) como um conjunto de validação adicional — hoje o treinamento é 100% cego ao comportamento físico real.
- **Ensemble de genomas.** Em vez de embutir um único "melhor genoma", treinar e embutir vários, e deixar o sistema real votar entre eles — reduz o risco de um único genoma ter um ponto fraco não detectado pelos cenários de treino.
- **Aprendizado contínuo supervisionado pelo operador.** Permitir que o operador, durante o uso real, marque uma decisão da rede como "errada" — e usar esse sinal (mesmo que esparso) para re-treinar periodicamente, em vez de o genoma só mudar manualmente a cada nova sessão de treinamento na tela.

7 Glossário rápido

Genoma
Um vetor de números (pesos da rede neural) — um "indivíduo" candidato dentro do algoritmo genético.

Fitness
A nota de qualidade de um genoma, calculada rodando episódios simulados completos.

Geração
Uma rodada completa de avaliação + seleção + cruzamento + mutação de toda a população de genomas.

Heurística
Regra fixa escrita à mão (ex.: "prefira a torre do mesmo quadrante"), sem nenhum aprendizado.

Controlador híbrido
A combinação de heurística + ajuste da rede neural, usada tanto na simulação de treino quanto no sistema real.

Cenário tático catalogado
Um caso de teste nomeado e reprodutível (enxame, disputa de torre etc.), usado para forçar a rede a enfrentar situações específicas a cada geração.

Cenário personalizado
Um episódio desenhado à mão pelo usuário na tela de treinamento — posição e velocidade de cada alvo definidas manualmente, sem sorteio.